

BEES Data Engineering – Breweries Case

You work as a Data Engineer at a large brewing company, and your mission is to build a **resilient and scalable Data Lake from scratch**. To accomplish this, you will consume data from the [Open Brewery DB API](#), which provides information about breweries and pubs around the world.

To guide your work, the Data Architect has left the following **instructions**:

- **Architecture:** Describe how you would design the data architecture. Provide a **blueprint or documentation** that represents your proposed solution and explains the main components and data flow.
- **Data Lake Architecture:** Your data lake must follow the medallion architecture having a bronze, silver, and gold layer:
 - a) **Bronze Layer:** Persist the raw data from the API in its native format or any format you find suitable.
 - b) **Silver Layer:** Transform the data to a columnar storage format such as parquet or delta, and partition it by brewery location. Please explain any other transformations you perform.
 - c) **Gold Layer:** Create an aggregated view with the quantity of breweries per type and location.
- **Language:** Use the language of your preference for the requests and data transformation. Please include test cases for your code. Python and PySpark are preferred.
- **Orchestration Tool:** To monitor your pipelines and schedule executions continuously choose the orchestration tool of your preference (Airflow, Luigi, Mage etc.) to build a data pipeline. We're interested in seeing your ability to handle scheduling, retries, and error handling in the pipeline.
- **Data Quality:** Describe how you would design and implement data quality checks, detailing the techniques or rules you would apply and what dimensions of data quality these checks would help ensure.
- **Observability / Alerts:** Describe how you would design monitoring and alerting mechanisms to ensure pipeline reliability — including how you would track pipeline performance, detect and diagnose failures, and notify the appropriate stakeholders when incidents occur.

Extra points:

If you feel comfortable, you may implement the following optional features to earn **extra points** in your evaluation. However, remember to stay focused on the main objectives of the challenge.

- a) **Deployment:** Show how your solution could be deployed or automated using Docker or Kubernetes for modularization.

Delivery method:

- **Repository:** Create a public repository on GitHub with your solution. Include our code and documentation about design choices, trade-offs, and provide clear instructions on how to run your application.
- **Cloud Services:** If your solution requires any cloud services, please provide instructions on how to set them up. Please do not post them in your public repository.

Evaluation Criteria:

Your solution will be evaluated based on the following criteria:

1. Solution Design
2. Efficiency
3. Code Quality
4. Completeness
5. Documentation
6. Error Handling

Time Frame:

Please complete the test within 1 week and share the link to your GitHub repository with us. Remember, the goal of this test is to showcase your skills and approach to building a data pipeline. Good luck!